

Bio Informatics

Lecture 16

Multiple Sequence Alignment – Algorithms

- Given k sequences $S = \{S_1, S_2, \dots, S_k\}$.
- A multiple alignment of S is a set of k equal-length sequences $\{S'_1, S'_2, \dots, S'_k\}$.
 - where S'_i is obtained by inserting gaps in to S_i .
- The multiple sequence alignment problem aims to
 - find a multiple alignment which optimize certain score.

Multiple Sequence Alignment – Algorithms

- S1= ACGGAGA
- S2= CGTTGACA
- S3= ACTGAA
- S4= CCGTTCAC

Multiple Sequence Alignment – Algorithms

- S1= ACG--GAGA
- S2= -CGTTGACA
- S3= AC-T-GA-A
- S4= CCGTTCAC-

Multiple Sequence Alignment – Why?

- Align the domains of proteins
- Align the same genes/proteins from multiple species
- Help predicting protein structure

Sum-Of-Pair (SP) Score - MSA

- Consider the multiple alignment S' of S .
- $\text{SP-score}(a_1, \dots, a_k) = \sum_{1 \leq i < j \leq k} \delta(a_i, a_j)$
 - where a_i can be any character or a space.
- The SP-score of S' is
 - $\sum_x \text{SP-score}(S'_1[x], \dots, S'_k[x])$.

MSA of 4 sequences and SP-Scores

- $S_1 = \text{ACG--GAGA}$
- $S_2 = \text{-CGTTGACA}$
- $S_3 = \text{AC-T-GA-A}$
- $S_4 = \text{CCG TTCAC-}$
- Assume score of
 - match and mismatch/insert/delete are 2 and -2, respectively.
- For position 1,
 - $\text{SP-score}(A,-,A,C) = 2\delta(A,-) + 2\delta(A,C) + \delta(A,A) + \delta(C,-) = -8$
- $\text{SP-score} = -8 + 12 + 0 + 0 - 6 + 0 + 12 - 10 + 0 = 0$

Sum-Of-Pair (SP) Distance - MSA

- Equivalently, we have SP-dist.
- Consider the multiple alignment S' of S .
- $\text{SP-dist}(a_1, \dots, a_k) = \sum_{1 \leq i < j \leq k} \delta(a_i, a_j)$
 - where a_i can be any character or a space.
- The SP-dist of S' is
 - $\sum_x \text{SP-dist}(S'_1[x], \dots, S'_k[x])$.

MSA – To Do List

- Exact result
 - Dynamic Programming
- Approximation algorithm
 - Center star method
- Heuristics
 - ClustalW --- Progressive alignment
 - MUSCLE --- Iterative method

MSA – Dynamic Programming

- Recall that the optimal alignment for two sequences can be found as follows.
- Let $V(i_1, i_2)$ be the score of the optimal alignment between $S_1[1..i_1]$ and $S_2[1..i_2]$.

$$V(i_1, i_2) = \max \begin{cases} V(i_1 - 1, i_2 - 1) + \delta(S_1[i_1], S_2[i_2]) \\ V(i_1 - 1, i_2) + \delta(S_1[i_1], _) \\ V(i_1, i_2 - 1) + \delta(_, S_2[i_2]) \end{cases}$$

- The equation can be rephased as

$$V(i_1, i_2) = \max_{(b_1, b_2) \in \{0,1\}^2 - \{(0,0)\}} \{V(i_1 - b_1, i_2 - b_2) + \delta(S_1[i_1 b_1], S_2[i_2 b_2])\}$$

MSA – Dynamic Programming

- Dynamic programming is expensive in both time and space. It is rarely used for aligning more than 3 or 4 sequences.

MSA – Center Star Alignment

- Computing optimal multiple alignment takes exponential time.
- Can we find a good approximation using polynomial time?
- We introduce Center star method, which minimizes Sum-of-Pair distance.

Center Star Algorithm – Example

- S1 : CCTGCTGCAG
- S2 : GATGTGCCG
- S3 : GATGTGCAG
- S4 : CCGCTAGCAG
- S5 : CCTGTAGG

Center Star – Distance Matrix (Step-1)

	S_1	S_2	S_3	S_4	S_5
S_1	0	4			
S_2		0			
S_3			0		
S_4				0	
S_5					0

S_1 : CCTGCTGCAG

S_2 : GATG-TGCCG

S_1 : CCTGCTGCAG

S_3 : GATG-TGCAG

S_1 : CCTGCT-GCAG

S_4 : CC-GCTAGCAG

S_1 : CCTGCT-GCAG

S_5 : CCTG-TAG--G

Center Star – Distance Matrix (Step-1)

	S_1	S_2	S_3	S_4	S_5
S_1	0	4	3	2	4
S_2		0			
S_3			0		
S_4				0	
S_5					0

S_1 : CCTGCTGCAG

S_2 : GATG-TGCCG

S_1 : CCTGCTGCAG

S_3 : GATG-TGCAG

S_1 : CCTGCT-GCAG

S_4 : CC-GCTAGCAG

S_1 : CCTGCT-GCAG

S_5 : CCTG-TAG--G

Center Star – Distance Matrix (Step-1)

	S_1	S_2	S_3	S_4	S_5
S_1	0	4	3	2	4
S_2		0	1	6	5
S_3			0	5	5
S_4				0	4
S_5					0

S_1 : CCTGCTGCAG

S_2 : GATG-TGCCG

S_1 : CCTGCTGCAG

S_3 : GATG-TGCAG

S_1 : CCTGCT-GCAG

S_4 : CC-GCTAGCAG

S_1 : CCTGCT-GCAG

S_5 : CCTG-TAG--G

Center Star – SP-Distance (Step-2)

	S_1	S_2	S_3	S_4	S_5
S_1	0	4	3	2	4
S_2		0	1	6	5
S_3			0	5	5
S_4				0	4
S_5					0

$$\sum_{i=1..k} D(S_1, S_i) =$$

$$\sum_{i=1..k} D(S_2, S_i) =$$

$$\sum_{i=1..k} D(S_3, S_i) =$$

$$\sum_{i=1..k} D(S_4, S_i) =$$

$$\sum_{i=1..k} D(S_5, S_i) =$$

Center Star – SP-Distance (Step-2)

	S_1	S_2	S_3	S_4	S_5
S_1	0	4	3	2	4
S_2		0	1	6	5
S_3			0	5	5
S_4				0	4
S_5					0

$$\sum_{i=1..k} D(S_1, S_i) = 13$$

$$\sum_{i=1..k} D(S_2, S_i) = 16$$

$$\sum_{i=1..k} D(S_3, S_i) = 14$$

$$\sum_{i=1..k} D(S_4, S_i) = 17$$

$$\sum_{i=1..k} D(S_5, S_i) = 18$$

Center Star – Center S_c (Step-3&4)

S_1 : CCTGCTGCAG

S_2 : GATG-TGCCG

S_1 : CCTGCTGCAG

S_3 : GATG-TGCAG

S_1 : CCTGCTGCAG

S_2 : GATG-TGCCG

S_3 : GATG-TGCAG

S_1 : CCTGCTGCAG

S_2 : GATG-TGCCG

S_3 : GATG-TGCAG

S_1 : CCTGCT-GCAG

S_4 : CC-GCTAGCAG

S_1 : CCTGCTG-CAG

S_2 : GATG-TG-CCG

S_3 : GATG-TG-CAG

S_4 : CC-GCTAGCAG

S_1 : CCTGCT-GCAG

S_5 : CCTG-TAG--G

S_1 : CCTGCT-GCAG

S_2 : GATG-T-GCCG

S_3 : GATG-T-GCAG

S_4 : CC-GCTAGCAG

S_5 : CCTG-TAG--G

Center Star Algorithm – Stepwise

Center_Star_Method

Require: A set S of sequences

Ensure: A multiple alignment of M with sum of pair distances at most twice that of the optimal alignment of S

- 1: Find $D(S_i, S_j)$ for all i, j .
- 2: Find the center sequence S_c which minimizes $\sum_{i=1}^k D(S_c, S_i)$.
- 3: For every $S_i \in S - \{S_c\}$, choose an optimal alignment between S_c and S_i .
- 4: Introduce spaces into S_c so that the multiple alignment $\mathcal{M}$ satisfies the alignments found in Step 3.

Center Star Algorithm – Running Time (Optimal Approx.)

- Assume all k sequences are of length n .
 - Step 1 takes $O(k^2n^2)$ time.
 - Step 2 takes $O(k^2)$ time to find the center string S_c .
 - Step 3 takes $O(kn^2)$ time to compute the alignment between S_c and S_i for all i .
 - Step 4 introduces space into the multiple alignment, which takes $O(k^2n)$ time.
- In total, the running time is $O(k^2n^2)$.

S_1 : CCTGCTGCAG
 S_2 : GATGTGCCG
 S_3 : GATGTGCAG
 S_4 : CCGCTAGCAG
 S_5 : CCTGTAGG

	S_1	S_2	S_3	S_4	S_5
S_1	0	4	3	2	4
S_2		0	1	6	5
S_3			0	5	5
S_4				0	4
S_5					0

Step 1
 $O(k^2n^2)$

$\sum_{i=1..k} D(S_1, S_i) = 13$
 $\sum_{i=1..k} D(S_2, S_i) = 16$
 $\sum_{i=1..k} D(S_3, S_i) = 14$
 $\sum_{i=1..k} D(S_4, S_i) = 17$
 $\sum_{i=1..k} D(S_5, S_i) = 18$

Step 2
 $O(k^2)$

S_1 : CCTGCTGCAG
 S_2 : GATG-TGCCG

S_1 : CCTGCTGCAG
 S_3 : GATG-TGCAG

S_1 : CCTGCT-GCAG
 S_4 : CC-GCTAGCAG

S_1 : CCTGCT-GCAG
 S_5 : CCTG-TAG--G

Step 3
 $O(kn^2)$

S_1 : CCTGCT-GCAG
 S_2 : GATG-T-GCCG
 S_3 : GATG-T-GCAG
 S_4 : CC-GCTAGCAG
 S_5 : CCTG-TAG--G

Step 4
 $O(k^2n)$